

C# 列印範本與版型修改教學

非資訊背景同事適用版

文件用途：協助沒有程式設計背景的同事，理解列印範本的基本概念，安全完成文字、欄位、圖片、條碼、QR Code、明細列與條件顯示等常見修改。

適用對象：門市、客服、營運、倉儲、財務、測試與日常需要調整列印格式的人員。

建議授課時間：半天入門課加半天實作課。若同事平時很少接觸 JSON，建議拆成兩次課程。

重要原則：同事學習的是「修改範本」，不是「修改 C# 程式」。C# 解析器是負責讀取範本與資料的固定引擎；一般使用者只需要在規定的範本欄位中修改內容，並透過預覽與測試列印確認結果。

一、課程完成後應具備的能力

完成本課程後，學員應能夠：

能力	學員應能完成的工作
認識範本	說明 Page、Text、Rows、Image、QrCode、BarCode 等元件用途
修改文字	修改固定標題、文字內容、字體大小、粗體、對齊與位置
綁定資料	使用既有資料欄位，例如 <code>{ \$OrderNo }</code> 、 <code>{ \$CustomerName }</code>
修改明細	調整訂單明細欄位、欄位順序與每列顯示內容
修改圖片	更換 Logo 或其他既有圖片資料來源
修改條碼	調整條碼種類、內容欄位與位置
修改 QR Code	調整 QR Code 內容欄位、大小與錯誤修正等級
使用條件	讓特定區塊只在有資料或符合條件時列印
預覽驗證	使用測試資料檢查內容、位置、換行與列數
故障排除	判斷是資料問題、範本問題、版面問題或印表機問題

安全修改

建立副本、保留版本、記錄變更，不直接覆蓋正式範本

學員**不需要**學會 C#、Visual Studio、類別、方法、例外處理或資料庫操作。這些屬於解析器維護人員的工作。

二、先建立正確觀念

2.1 三個角色

整個列印系統可以分成三個部分：

部分	白話說明	誰負責
資料	要列印的內容，例如訂單編號、商品名稱、數量與金額	業務系統或資料提供者
範本	決定內容放在哪裡、長什麼樣子	範本維護人員
解析器	讀取資料與範本，產生列印結果	C# 程式與開發人員

可以用以下方式向同事說明：

資料決定「印什麼」，範本決定「怎麼印」，解析器負責「把兩者組合起來」。

如果範本沒有寫錯，但資料本身沒有提供某個欄位，就不會有內容可以列印。如果資料有提供欄位，但範本沒有引用它，也不會自動出現在列印結果中。

2.2 範本不是 Word 文件

範本通常以 JSON 格式保存。它不像 Word 一樣可以直接看到排版結果，而是用欄位描述元件。

例如：

JSON

```
{
  "ElementType": "Text",
  "X": 10,
  "Y": 8,
  "Width": 60,
  "Height": 5,
```

```
"Content": "訂單編號 : {$OrderNo}"
}
```

這段內容的意思是：

設定	意義
ElementType	這是一個文字元件
X	從左邊開始的水平位置
Y	從上方開始的垂直位置
Width	元件可使用的寬度
Height	元件可使用的高度
Content	實際要顯示的文字與資料欄位

2.3 什麼是 JSON

JSON 是一種用文字表示資料的格式。學員只需要先認識四個符號：

符號	用途	範例
{ }	一組設定的開始與結束	{ "X": 10 }
[]	多筆資料的開始與結束	[{ ... }, { ... }]
:	欄位名稱與內容的分隔	"X": 10
,	同一層的下一個欄位或元件	"X": 10, "Y": 8

最常見的錯誤是漏掉逗號、漏掉引號，或把大括號刪掉。因此每次修改後都必須先執行範本檢查，再進行列印。

三、課程安排與教學方法

3.1 建議課程表

單元	主題	建議時間	教學方式
----	----	------	------

1	系統概念與安全操作	30 分鐘	講解與示範
2	看懂範本基本結構	45 分鐘	對照範例
3	修改文字與欄位	60 分鐘	示範加練習
4	調整位置、大小與對齊	45 分鐘	預覽實作
5	修改明細列 Rows	60 分鐘	訂單範例
6	圖片、條碼與 QR Code	45 分鐘	元件實作
7	條件顯示與嵌入頁面	45 分鐘	情境練習
8	測試與故障排除	45 分鐘	錯誤案例
9	綜合實作與驗收	60 分鐘	個人操作

3.2 教學時的說明順序

每一個功能都按照以下順序說明：

1. 這個功能解決什麼問題。
2. 使用者要修改哪一個欄位。
3. 修改前後的範例。
4. 預期的列印結果。
5. 最常見的錯誤。
6. 如何預覽與驗證。

不要一開始就講 C# 類別名稱、解析流程或內部演算法。對非資訊背景同事而言，先學會「看懂設定、修改設定、確認結果」比了解程式內部實作更重要。

四、單元一：安全操作與工作流程

4.1 正式修改的標準流程

每次修改範本都必須依照以下流程：

Plain Text

複製正式範本



重新命名為測試版本



只修改一個目的



儲存



執行格式檢查



使用固定測試資料預覽



實機測試列印



請第二人確認



記錄版本與修改內容



才替換正式版本

4.2 範本檔案命名規則

建議採用以下格式：

Plain Text

[用途]_[紙寬]_[版本]_[日期].json

例如：

Plain Text

Order_80mm_v1.0_20260911.json
Invoice_57mm_v1.2_20260911.json
Kitchen_80mm_v2.0_Test.json

正式檔案與測試檔案必須分開保存。 **Test**、**Backup** 或日期版本檔案不可直接當作正式檔案使用，除非已完成驗收。

4.3 不可直接修改的內容

非開發人員原則上不可修改以下內容：

- C# 解析器程式。
- 印表機驅動程式。
- 紙張寬度的核心設定。

- DPI 或縮放的全域設定。
- `ElementType` 的拼字。
- 系統約定的資料欄位名稱。
- `IncludePage` 的循環嵌入設定。
- `CommandMode` 的印表機控制指令。

如果需要修改以上內容，應提出需求給解析器維護人員處理。

五、單元二：認識範本的基本結構

5.1 Page 是一張列印頁

`Page` 代表一張完整的列印版面。

JSON

```
{
  "ElementType": "Page",
  "X": 0,
  "Y": 0,
  "Width": 80,
  "Height": 50,
  "ChildElements": []
}
```

`ChildElements` 代表這張頁面裡面的所有元件。文字、圖片、條碼和明細都放在其中。

5.2 元件類型對照表

ElementType	用途	日常修改頻率
Page	定義整張紙的範圍與基本設定	低
Text	固定文字或資料欄位	高
Rows	重複列印多筆陣列資料	高
Image	Logo 或圖片	中
QrCode	QR Code	中

BarCode	一維條碼	中
Table	有資料才顯示的區塊	中
Block	單一系列或區塊	中
IncludePage	引用其他頁面或子範本	低
CommandMode	送出特殊印表機指令	很低

5.3 元件的共同欄位

欄位	用途	初學者建議
X	水平位置	先用預覽微調
Y	垂直位置	先用預覽微調
Width	寬度	文字太長時增加
Height	高度	多行文字時增加
Content	顯示內容	最常修改
ElementType	元件類別	不要任意修改
X_Alignment	水平定位方式	使用既有範例
Y_Alignment	垂直排列方式	使用既有範例
Index	顯示順序	不要任意重排

六、單元三：修改固定文字與資料欄位

6.1 固定文字

固定文字直接寫在 Content 裡：

JSON

```
"Content": "歡迎光臨"
```

修改為：

```
JSON
```

```
"Content": "謝謝您的光臨"
```

固定文字不會隨訂單資料改變。

6.2 資料欄位

資料欄位通常使用以下格式：

```
Plain Text
```

```
{ $欄位名稱 }
```

範例：

```
JSON
```

```
"Content": "訂單編號 : { $OrderNo }"
```

如果資料如下：

```
JSON
```

```
{  
  "OrderNo": "A20260911001"  
}
```

列印結果為：

```
Plain Text
```

```
訂單編號 : A20260911001
```

6.3 同一行放置多個欄位

```
JSON
```

```
"Content": "{$ProductName} {$Quantity} 件 $ {$Amount}"
```

欄位之間可以使用空格、全形空格或固定文字分隔，但不要自行改變欄位名稱。

6.4 資料欄位名稱不能自行猜測

以下兩個名稱是不同欄位：

Plain Text

```
{$OrderNo}  
{$OrderNumber}
```

即使人類看起來意思相近，解析器也會將它們視為不同名稱。修改前應查閱「資料欄位清單」，不要自行創造欄位名稱。

6.5 文字修改練習

請將以下範本：

JSON

```
{  
  "ElementType": "Text",  
  "Content": "客戶：{$Name}"  
}
```

修改成以下格式：

Plain Text

會員姓名：王小明
電話：0912-345-678

參考答案：

JSON

```
{  
  "ElementType": "Text",  
  "Content": "會員姓名：{$Name}\n電話：{$Phone}"  
}
```

若目前解析器不支援 `\n`，則應使用系統既有的換行方式，不能自行假設。課堂上必須以實際解析器的測試結果為準。

七、單元四：位置、大小、對齊與換行

7.1 位置的基本概念

範本的座標通常以左上角為起點：

Plain Text

(0,0)

————→ X 增加

|

|

↓

Y 增加

調整規則如下：

想要的效果	應調整的欄位
往右移	增加 <code>X</code>
往左移	減少 <code>X</code>
往下移	增加 <code>Y</code>
往上移	減少 <code>Y</code>
顯示較寬	增加 <code>Width</code>
顯示較高	增加 <code>Height</code>

7.2 不要一次修改太多數值

正確做法是每次只調整一個方向。例如：

Plain Text

第一次：`X + 2`

第二次：`Y - 1`

第三次：Width + 5

錯誤做法是一次同時修改 X、Y、Width、Height、字體大小，之後無法判斷哪一個設定造成問題。

7.3 字體與粗體

文字元件常見設定包括：

JSON

```
"ContentSize": 13,  
"ContentBold": "Y"
```

設定	效果
增加 <code>ContentSize</code>	文字變大
減少 <code>ContentSize</code>	文字變小
<code>ContentBold: "Y"</code>	粗體
<code>ContentBold: "N"</code>	一般字

字體大小變大後，可能需要同時增加 `Width` 或 `Height`。

7.4 對齊

常見設定如下：

JSON

```
"HorizontalContentAlign": "Left"
```

可用值通常包括：

Plain Text

Left	靠左
Center	置中
Right	靠右

垂直方向可能包括：

Plain Text	
Top	靠上
Center	垂直置中
Down	靠下

欄位名稱請以目前解析器的實際規格為準。若範本中已使用 `HorizontalContentAlig`，不要自行改成看似較正確的 `HorizontalContentAlign`，因為解析器可能只認得前者。

7.5 自動換行

常見設定：

JSON

```
"AutoWrap": "Y",
"WrapMaxRows": 2,
"SmallContentSize": 9
```

意思是：

- 內容太長時允許換行。
- 最多顯示兩行。
- 超過限制時縮小文字。

常見問題如下：

問題	可能原因	建議處理
文字被截斷	<code>AutoWrap</code> 為 <code>N</code>	改用 <code>Y</code> 或增加寬度
文字壓到下一欄	<code>Width</code> 太大或位置重疊	減少寬度或調整 <code>X</code>
文字超出紙張	<code>X</code> 加 <code>Width</code> 超過紙寬	調整 <code>X</code> 或 <code>Width</code>
多行後蓋住明細	<code>Height</code> 太小	增加高度或設定最大行數

八、單元五：修改訂單明細 Rows

8.1 Rows 的用途

Rows 用來處理一筆以上的資料，例如：

- 訂單商品明細。
- 發票明細。
- 配送品項。
- 付款項目。
- 廚房出單品項。

它會依照陣列中的資料筆數重複產生列印內容。

8.2 基本結構

JSON

```
{
  "ElementType": "Rows",
  "RootName": "Items",
  "DisplayMode": "Single",
  "ChildElements": [
    {
      "ElementType": "Text",
      "Content": "${Name}"
    },
    {
      "ElementType": "Text",
      "Content": "${Qty}"
    },
    {
      "ElementType": "Text",
      "Content": "${Amount}"
    }
  ]
}
```

其中 RootName: "Items" 表示要讀取 Items 陣列。

8.3 資料與結果

資料：

JSON

```
{
  "Items": [
```

```
{ "Name": "咖啡", "Qty": 2, "Amount": 120 },  
  { "Name": "蛋糕", "Qty": 1, "Amount": 80 }  
]
```

預期結果：

Plain Text

```
咖啡  2  120  
蛋糕  1   80
```

8.4 Single 與 Merge

模式	意義	適用情境
Single	每筆資料各自形成一列	訂單明細、廚房出單
Merge	多筆相同欄位合併於同一列或同一元件	標籤、簡短品項集合

初學者只需要修改 `ChildElements` 裡面的文字內容。除非已確認解析器行為，否則不要自行更換 `DisplayMode`。

8.5 明細欄位調整範例

原本：

JSON

```
"Content": "{$Name}  {$Qty}"
```

修改為：

JSON

```
"Content": "{$Name}  x{$Qty}  ${$Amount}"
```

修改前必須確認：

1. `Amount` 是資料中真實存在的欄位。
2. 明細欄位的寬度足以顯示金額。

3. 商品名稱太長時不會蓋住數量。
4. 最長商品名稱已經用測試資料驗證。

8.6 空明細的處理

如果陣列沒有資料，可能出現以下結果：

- 整個明細區塊不顯示。
- 仍顯示表頭但沒有明細。
- 顯示空白列。
- 解析器回報錯誤。

這是由 `Table` 的 `AlwaysPrint`、條件設定或解析器規則決定。不要只靠猜測，應使用「空陣列測試資料」驗證。

九、單元六：圖片、QR Code 與 Barcode

9.1 圖片與 Logo

圖片常用於 Logo。範本通常引用資料欄位，例如：

JSON

```
{
  "ElementType": "Image",
  "Content": "${LogoFile}",
  "X": 5,
  "Y": 5,
  "Width": 30,
  "Height": 11
}
```

圖片顯示不出來時，依序檢查：

1. `LogoFile` 是否真的有資料。
2. 資料是否為解析器支援的 Base64 格式。
3. `Width` 與 `Height` 是否大於零。
4. 圖片位置是否超出紙張。
5. 圖片是否被其他元件蓋住。

9.2 QR Code

QR Code 常用於：

- 發票查詢網址。
- 付款資訊。
- 訂單查詢網址。
- 會員或票券資訊。

基本概念如下：

JSON

```
{
  "ElementType": "QrCode",
  "Content": "{$QrContent}",
  "ErrorCorrection": "H",
  "X": 50,
  "Y": 10,
  "Width": 25,
  "Height": 25
}
```

錯誤修正等級常見為：

等級	可容許的損壞比例	建議
L	約 7%	內容短且列印品質穩定
M	約 15%	一般情況
Q	約 25%	可能有輕微污損
H	約 30%	重要資訊或可能受損的標籤

QR Code 太小、內容太長或印表機解析度不足時，可能造成手機無法掃描。修改後必須用實際手機掃描，不可只看外觀。

9.3 一維條碼

基本結構：

JSON

```
{
  "ElementType": "BarCode",
  "Format": "CODE_128",
  "Content": "{$BarcodeContent}",
  "X": 5,
  "Y": 20,
  "Width": 60,
  "Height": 15
}
```

常見條碼類型包括：

- CODE_128
- CODE_39
- EAN_13
- EAN_8
- UPC_A
- DATA_MATRIX

條碼類型不能隨意替換。不同格式對內容長度、數字或英文字元有不同要求。變更條碼格式時，必須由負責系統的人確認。

十、單元七：條件顯示與 IncludePage

10.1 為什麼需要條件顯示

同一份範本可能要處理不同情況：

- 有發票才顯示發票區域。
- 有 Logo 才顯示 Logo。
- 有電子票證資料才顯示電子票證欄位。
- 有折扣才顯示折扣列。
- 有商品明細才顯示明細表。

10.2 條件判斷的基本概念

條件設定通常包括：

Plain Text

ConditionalKey
Conditional
ConditionalValue

例如：

JSON

```
{  
  "ConditionalKey": "PrintInvoice",  
  "Conditional": "==",  
  "ConditionalValue": "Y"  
}
```

意思是：只有 `PrintInvoice` 的值等於 `Y` 時，才顯示這個區塊。

10.3 常見判斷方式

判斷符號	意義	範例
空白	判斷是否有資料	欄位有內容才顯示
<code>==</code>	等於	<code>PrintInvoice == Y</code>
<code>!=</code>	不等於	<code>Status != Cancel</code>
<code>></code>	大於	<code>Qty > 0</code>
<code><</code>	小於	<code>Amount < 1000</code>
<code>>=</code>	大於或等於	<code>Qty >= 1</code>
<code><=</code>	小於或等於	<code>Amount <= 1000</code>

初學者最容易犯的錯誤，是把 `Y`、`N`、`true`、`false` 混用。應以資料來源與現有範本的約定為準。

10.4 IncludePage

`IncludePage` 可以把另一個已建立的頁面嵌入目前頁面。例如：

Plain Text

主訂單頁

- └─ 訂單標題
- └─ 商品明細
- └─ 嵌入發票頁
- └─ 嵌入電子票證頁

使用 IncludePage 時要注意：

- 被引用的頁面名稱或 SID 必須正確。
- 不要讓頁面互相引用造成無限循環。
- 被引用頁面的條件資料必須存在。
- 修改後要測試「有資料」與「無資料」兩種情況。

一般使用者只應調整已經建立好的 IncludePage 參數，不應自行建立多層循環嵌入。

十一、單元八：紙張、DPI 與印表機設定

11.1 Pixel 與毫米

範本可能使用 pixel 作為座標，而紙張通常使用毫米表示。基本換算為：

Plain Text

$$\begin{aligned}\text{毫米} &= \text{pixel} \div \text{DPI} \times 25.4 \\ \text{pixel} &= \text{毫米} \div 25.4 \times \text{DPI}\end{aligned}$$

目前資料以 203 DPI 為主要換算基準。若印表機 DPI 不同，不能直接沿用相同數值。

11.2 常見紙寬

紙張	常見用途	注意事項
57mm	小型收據或標籤	可用寬度較小，容易截斷
80mm	收據、發票、一般熱感列印	需確認驅動設定為 80mm
其他寬度	特殊標籤	必須有專用範本與驅動設定

11.3 紙張設定的安全規則

試算表中的經驗規則必須納入課堂：

- DT-2205 如果紙張設定過小，可能導致無法列印或紅燈閃爍。
- 某些 80mm 熱感印表機若設定成 57mm，列印內容可能被截斷。
- 紙張寬度必須同時符合範本、驅動程式與實際裝入的紙張。
- 紙張間隔高度可能由印表機驅動程式控制。
- 修改紙寬或驅動設定前，必須由設備管理人員確認。

十二、單元九：測試與驗收

12.1 每次修改的最小測試集

每次變更至少要測試以下情況：

測試情境	要確認的內容
一般短文字	是否正常顯示與對齊
最長文字	是否換行、截斷或蓋住其他欄位
空白欄位	是否留下不必要的標籤或空白
一筆明細	Rows 是否正常顯示
多筆明細	是否正確重複與排列
零筆明細	是否符合預期顯示規則
有條件資料	條件區塊是否出現
無條件資料	條件區塊是否隱藏
條碼掃描	是否能被掃描器讀取
QR Code 掃描	是否能被手機讀取
實體列印	是否超出紙張或被截斷

12.2 驗收檢查表

Plain Text

- ☐ 已建立測試版本副本
- ☐ 已記錄修改目的
- ☐ JSON 格式檢查通過
- ☐ 測試資料欄位名稱正確
- ☐ 固定文字正確
- ☐ 文字位置正確
- ☐ 文字沒有互相重疊
- ☐ 長文字測試通過
- ☐ 空白欄位測試通過
- ☐ 多筆明細測試通過
- ☐ QR Code 已實際掃描
- ☐ Barcode 已實際掃描
- ☐ 實體列印尺寸正確
- ☐ 57mm／80mm 紙張設定正確
- ☐ 已由第二人檢查
- ☐ 已更新版本與變更紀錄

12.3 變更紀錄範例

日期	範本	版本	修改者	修改內容	測試結果
2026-09-11	Order_80mm	v1.1	王小明	增加會員電話	通過
2026-09-11	Invoice_57mm	v1.2	李小華	調整 QR Code 位置	通過

十三、單元十：故障排除教學

13.1 先判斷問題類型

遇到問題時，先歸類，不要立刻修改很多設定：

現象	優先懷疑的類型
範本完全無法載入	JSON 格式或範本結構
某個欄位空白	資料欄位名稱或資料內容
文字位置錯誤	X、Y、Width、Height

文字被截斷	寬度、換行、紙寬或驅動
明細列數錯誤	Rows、RootName 或資料陣列
QR Code 無法掃描	內容、尺寸、錯誤修正或列印品質
Barcode 無法掃描	Format、內容格式或寬度
Logo 不顯示	Base64 資料、圖片欄位或尺寸
印表機紅燈	紙張、驅動或設備設定
列印結果左右偏移	紙張寬度、邊界或驅動校正

13.2 常見錯誤案例

案例一：欄位沒有顯示

範本：

```
JSON

"Content": "電話：{$PhoneNumber}"
```

但資料實際欄位為：

```
JSON

"Phone": "0912-345-678"
```

問題在於欄位名稱不一致。應改為：

```
JSON

"Content": "電話：{$Phone}"
```

案例二：範本無法解析

錯誤：

```
JSON

{
  "ElementType": "Text"
```

```
"Content": "Hello"
}
```

ElementType 後面少了逗號。正確寫法：

JSON

```
{
  "ElementType": "Text",
  "Content": "Hello"
}
```

案例三：文字超出紙張

設定：

Plain Text

```
X = 70
Width = 30
紙寬 = 80mm
```

X 加上 Width 已超過可用範圍。處理方式是減少 X、減少 Width，或改用較小字體。

案例四：明細只印一筆

可能原因包括：

- RootName 不是實際陣列名稱。
- 範本使用了 Block 而不是 Rows。
- 測試資料實際只有一筆。
- 條件設定限制了顯示筆數。

應先查看測試資料，再檢查 RootName，最後確認元件類型。

案例五：實機列印被截斷

依序檢查：

1. 範本紙寬是否正確。
2. 印表機驅動紙寬是否正確。
3. 實際裝入紙張是否相同。
4. X 加 Width 是否超過紙寬。

5. 印表機 DPI 與範本是否一致。
 6. 是否為 57mm 紙張套用了 80mm 範本。
-

十四、綜合實作課程

實作題目：修改一張 80mm 訂單收據

14.1 需求

將既有訂單範本修改為：

1. 頂端顯示店家名稱。
2. 顯示訂單編號與訂單日期。
3. 顯示客戶姓名。
4. 顯示商品名稱、數量與金額。
5. 有折扣時顯示折扣列。
6. 顯示訂單 QR Code。
7. 保留原本 Logo。

14.2 學員操作順序

1. 複製原始範本並建立新版本。
2. 找到 Page 元件。
3. 找到店家名稱的 Text 元件。
4. 修改固定文字，不修改 ElementType。
5. 找到訂單編號欄位，確認資料欄位名稱。
6. 修改 Rows 中的商品名稱、數量與金額。
7. 使用一筆、多筆與空明細資料進行預覽。
8. 找到折扣區塊，確認條件設定。
9. 調整 QR Code 位置與大小。
10. 儲存、檢查、預覽與實體列印。
11. 填寫驗收檢查表。

14.3 評量標準

評量項目	配分
正確建立版本副本	10
正確修改固定文字	10
正確引用資料欄位	20
正確修改 Rows	20
長文字與多筆資料測試	15
條件區塊測試	10
QR Code 掃描成功	5
實體列印無截斷與重疊	5
完成變更紀錄	5
總分	100

建議通過標準為 80 分，且不得有以下重大錯誤：

- 正式範本被直接覆蓋。
- 紙寬設定錯誤造成列印設備異常。
- 條碼或 QR Code 顯示錯誤但未測試。
- 範本包含無法解析的 JSON。

十五、提供給同事的一頁式速查表

15.1 最常修改的設定

需求	修改內容
改文字	Content
改資料欄位	{ \$欄位名稱 }
往右移	增加 X
往左移	減少 X

往下移	增加 Y
往上移	減少 Y
改寬度	Width
改高度	Height
改字體大小	ContentSize
改粗體	ContentBold
改靠左、置中、靠右	HorizontalContentAlig
改明細資料來源	Rows.RootName
改明細欄位	Rows.ChildElements.Content
改 QR Code 內容	QrCode.Content
改條碼格式	BarCode.Format

15.2 修改前必問自己五個問題

1. 我現在修改的是測試副本嗎？
2. 我修改的欄位名稱在資料中真的存在嗎？
3. 我有沒有誤刪逗號、引號或括號？
4. 文字變長後，其他元件會不會被推擠或蓋住？
5. 我有沒有用實際紙張與設備測試？

15.3 發生問題時提供給開發人員的資訊

回報問題時，請一次提供：

- 範本名稱與版本。
- 修改日期與修改者。
- 使用的測試資料。
- 預期結果。
- 實際結果。
- 預覽畫面或列印照片。
- 錯誤訊息全文。

- 使用的印表機名稱。
- 使用的紙張寬度。

不要只回報「印不出來」或「位置跑掉」，因為資訊不足時很難判斷問題來源。

十六、主管與講師的推行建議

16.1 建立分級權限

建議將範本使用者分成三個等級：

等級	能力範圍
L1 使用者	可修改固定文字與已指定的欄位內容
L2 範本維護者	可修改位置、尺寸、Rows、條件與圖片
L3 開發維護者	可修改解析器、資料模型、印表機指令與核心規則

一般同事先取得 L1 權限。完成實作與測驗後，再授予 L2 權限。

16.2 建立範本管理資料夾

建議資料夾結構：

Plain Text

```
Templates/  
├── Production/  
├── Testing/  
├── Backup/  
├── TestData/  
├── Preview/  
├── ChangeLog/  
└── Documentation/
```

正式版本只能放在 `Production`。測試中的檔案只能放在 `Testing`。這可以降低誤用測試版或誤刪正式版的風險。

16.3 建立範本變更申請單

變更申請至少應包括：

欄位	說明
申請人	提出需求的人
使用單位	哪個部門使用
範本名稱	要修改的範本
修改原因	為什麼需要改
修改項目	要改文字、欄位、位置或條件
預計上線日	何時使用
測試資料	用哪一筆資料驗證
驗收人	誰確認結果
回復方式	發生問題時如何還原

十七、建議的教學教材包

為了讓同事真正能夠獨立操作，正式上課前應準備以下檔案：

教材	內容
入門範本	只有 Page 與 Text，供第一次練習
訂單範本	包含 Rows 與商品明細
發票範本	包含條件與 IncludePage
圖片範本	包含 Logo 與 Base64 圖片
條碼範本	包含 QR Code 與 Barcode
測試資料	短文字、長文字、空白、多筆與極端資料
錯誤範本	故意放入漏逗號、錯欄位與超出紙寬的案例
速查表	一頁紙版本
驗收表	課後實作與正式上線使用
變更紀錄表	追蹤所有正式修改

十八、結語：希望同事形成的工作習慣

這套系統的學習重點不是記住所有 JSON 欄位，而是形成穩定的修改習慣：**先複製、少量修改、先檢查、再預覽、最後實機驗證。**

對非資訊背景同事而言，最重要的能力是知道哪些設定可以安全調整，哪些設定需要交給開發人員。只要資料欄位、元件類型、紙張設定與測試流程被管理好，大多數日常版型需求都可以由範本維護者獨立完成，而不需要每次都修改 C# 程式。

範本修改的完成標準不是「畫面看起來差不多」，而是「資料正確、位置正確、不同資料情境都正常、實體列印可用，而且有人可以追溯這次修改」。

References

[1] [Introducing JSON](#)

[2] [How to use System.Text.Json in .NET](#)

[3] [C# documentation](#)